컴파일러 과제**2** 보고서

20221500 양성준

1. LR 파싱 방법을 기반으로 하여 정수**, (,), +, *** 등으로 구성된 간단한 수식의 값을 계산하는 프로그램을 만들고 실험하시오**.**

1-1. 구현

1주차 강의노트 pdf의 29~36페이지 코드를 바탕으로 작성했습니다.
다음과 같은 사항을 추가했습니다.
- yylex 함수에서 여러 자리 정수를 하나의 NUMBER 토큰으로 인식하고 실제 정수값을 yylval에 저장
- 상태 stack과 semantic value stack을 Stack 구조체로 관리
- shift 시 토큰의 semantic value를 value stack에 함께 저장
- reduce 시 production rule에 따라 semantic action을 수행하여 수식값 계산

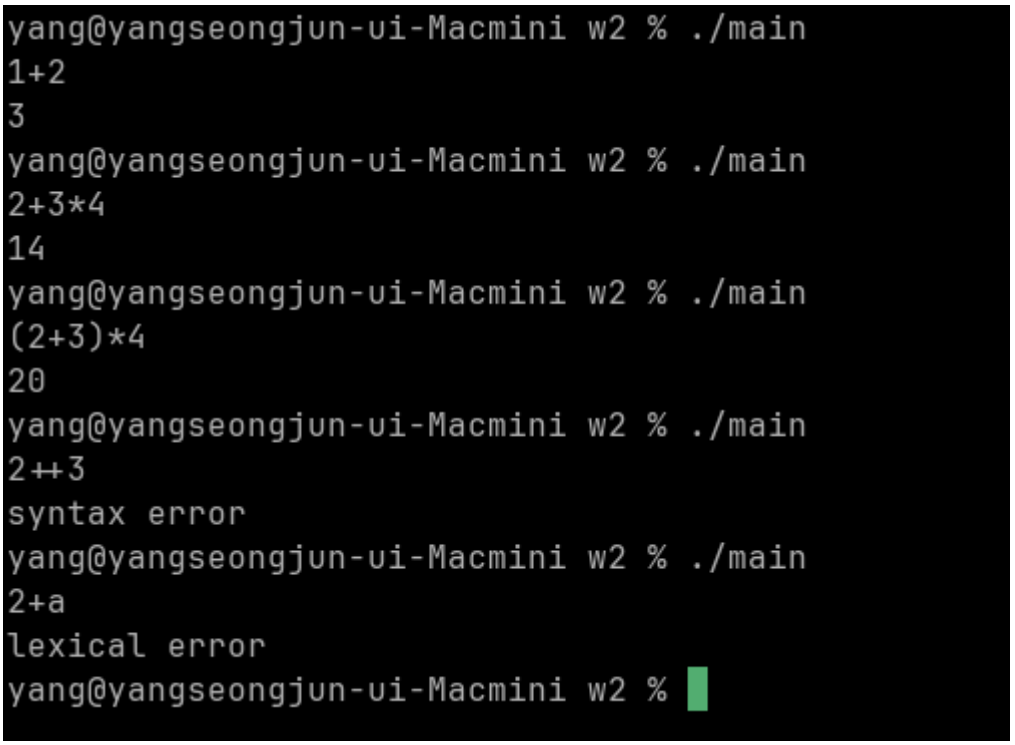
1-2. 검증

총 85개의 테스트 케이스를 구성하여 검증해봤습니다. 아래의 카테고리 별로 나누어서 확인했습니다.

카테고리	테스트 종류	예시	결과
기본 정상 수식	- 단일 정수, 여러 자리 정수 - 선행 0, +, * - 연속 덧셈/곱셈	1+2*3	모두 정상 계산
복합 정상 수식	- +, * 혼합 연산 및 연산자 우선순위 - 괄호, 중첩 괄호 - 공백/탭, 긴 수식	(1+2)*3	모두 정상 계산
연산자 관련 오류	- 연산자 단독 입력 - 앞/뒤 연산자 - 연속 연산자 (++, **, +*, *+)	1++2	syntax error
괄호 및 수식 구조 오류	- 빈 괄호 - 닫는 괄호 누락 혹은 초과 - 연산자 없이 피연산자가 연속되는 경우	(1+2	syntax error
미지원 입력	- -, / 연산자 - 영문자, 숫자 뒤 문자 - 실수 입력	1-2	lexical error

1-3. 실행 결과

UNIX 기반 환경에서 GCC를 사용하여 프로그램을 컴파일하고 실행했습니다.



1-4. 결론

이번 과제에서는 LR 파싱 방법을 이용하여 정수, 괄호, +, *로 구성된 수식을 분석하고 계산하는 프로그램을 구현했습니다.

Recursive-Descent Parser와 달리 LR Parser는 상태 stack과 ACTION/GOTO 테이블을 사용하여 shift와 reduce를 반복하는 bottom-up 방식으로 동작했습니다. 또한 reduce 과정에서 production rule별 semantic action을 수행하여 수식의 값을 계산했습니다.

처음에는 ACTION/GOTO 테이블과 상태 번호가 복잡하게 느껴졌지만, 실제 parser의 동작은 현재 상태와 입력 토큰을 이용해 테이블을 조회한 뒤 shift 또는 reduce를 수행하는 반복 구조라는 점을 이해할 수 있었습니다. 문법이 커질수록 파싱 테이블을 직접 관리하는 것은 복잡해질 수 있다는 점도 확인했습니다.

이 과정을 통해 ACTION/GOTO 테이블을 문법으로부터 어떤 원리로 기계적으로 생성하는지도 궁금해졌습니다.